CPSC 131: MIDTERM REVIEW

name: _____

Section 1. Efficiency Classes

Directions: Choose the most appropriate Big-O efficiency classes for the following ADT operations.

1. Advance a singly linked list iterator to the next location.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

2. Retrieve the last element from a singly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

3. Retrieve the front element from a singly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

4. Retrieve the last element from a doubly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

5. Remove all elements in a doubly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

6. pop_back() operation from a doubly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

7. Check if a singly linked list is empty.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

8. Insert an element into an ~~array~~ (vector).

a. O(1) b. O(1)* c. O(log n) d. O(n) e. O(n log n) f. $O(n^2)$

9. Push to a stack.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

10. Pop from a stack.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

11. Push to a linked-list based stack.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

12. Insert an element in a singly linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

13. Search for an element in a linked list.

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

14. Pop back from a Singly Linked List

a. O(1) b. O(log n) **c. O(n)** d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

Section 2. Efficiency Analysis

Directions: Choose the most appropriate Big-O efficiency classes for the following code snippets.

1.

```
for(int i = 0; i < n; ++i){
    std::cout << i * (i+1) << std::endl;
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

2.

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j)
        cout << "Complexity?";
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

3.

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n-2; ++j) {
        cout << "Row: " << i << " " << "Col: " << j << endl;
    }
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

4.Singly linked list sum function.

```
template <typename T>
int SinglyLinkedList<T>::calc_lst_sum() {
    int sum = 0;
    Node<T>* nodeptr = HEAD;
    while (nodeptr != nullptr) {
        sum += nodeptr->value;
        nodeptr = nodeptr->next;
    }
    return sum;
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

5. Singly linked list find function.

```
template <typename T>
bool SinglyLinkedList<T>::find(const T search_value) {
    Node<T>* nodeptr = HEAD;
        while (nodeptr != nullptr) {
            if (nodeptr->value == search_value)
                return true;
            nodeptr = nodeptr->next;
        }
    return false;
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(2^n)$

6.

```
for(int i = 0; i < n; ++i){
    for (int j = 0; j < n; ++j){
        for (int k = 0; k < n; ++k)
            std::cout << "Row: " << j << " " << "Col: " << k << std::endl;
    }
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(n^3)$

bonus:

```
for(int i = 0; i < n; ++i){
   i *= (i + 1);
}
```

a. O(1) b. O(log n) c. O(n) d. O(n log n) e. $O(n^2)$ f. $O(n^3)$

Section 3. Recursion

1. Write a recursive function, `recursive_sum()`, to sum all values less than and equal to the parameter (k). For example, if the input is the value 3, the function should return 6.

```
// write function here
3 + 2 + 1 + 0
7
7 + 6 + 5 + 4 + 3 + 2 + 1

100 -> 5050

3 steps in a recursive algorithm
   1) base case (check, logic, if)
   2) visit + (what are you doing at each recursive call) (cheat code: its the operation)
   3) recursive call (call itself, with updated parameters)
```

2. Write a function to recursively traverse a linked list and sums all the elements.

```
// finish the function
template <typename T>
int SLL<T>::sum(SNode<T> *head){
```

```
// start head, ending tail (nullptr)



}
```

3. Write a function to recursively traverse a linked list and return the total number of nodes.

```
// finish the function
template <typename T>
int SLL<T>::sizeRecur(SNode<T> *ptr){









}
```

Section 4. ADT Sketching

Directions: Sketch after each iteration & circle the final product of the Data Structure.

Exercise 1:

```
SinglyLinkedList<int> int_sll; // step 0
int_sll.push_front(2);       // step 1
int_sll.push_front(3);       // step 2
int_sll.push_front(4);       // step 3
while (!int_sll.empty()) {
    int_sll.remove_front();  // step 4
}
int_sll.push_front(1);       // step 5
```

steps:

final result:

Exercise 2:

```
DoublyLinkedList<int> int_dll; // step 0
int_dll.push_back(2);     // step 1
int_dll.push_front(3);        // step 2
int_dll.push_back(4);     // step 3
int_dll.push_back(1);     // step 4
int_dll.push_front(9);        // step 5
```

steps:

final result:

Exercise 3: VECTOR-BASED stack

```
stack<int> si; // step 0
si.push(1);    // step 1
si.push(2);    // step 2
si.push(3);    // step 3
si.pop();      // step 4
si.push(5);    // step 5
si.push(4);    // step 6
si.pop();      // step 7
```

steps:

final result:

Exercise 4: LIST-BASED stack

```
stack<int> fun;
fun.push(1);
fun.push(2);
fun.push(3);
fun.pop();
fun.push(5);
fun.push(4);
fun.pop();
```

steps:

final result:

Section 5. C++ Concepts

1.When is it better to allocate an object on the stack and when is it better to allocate an object on the heap?

2. Assume you are to write a ledger program that keeps track of each bank transaction as a struct containing the transaction type (deposit or withdrawal), the transaction amount (double), and the balance after the transaction. Would you use an array or a linked list? Why?

3. Assume you are to write a program to keep track of the customers signing in to be attended to. Would you use a stack or a queue? Why?

4. Explain how the capacity changes during Vector insertion?

5. Explain O(1) Amortized.

6. What are all the components within the Rule of 3?

7. Explain pass by reference vs pass by value. What is the purpose of passing by reference?

8. Identify and label the three (3) parts of the given recursive function

```
void printLinkedList(Node* head) {

  if (head == nullptr) return;

  std::cout << head->data << "\n";

  printLinkedList(head->next);

}
```

9. Write a function to print a singly linked list in reverse (iteratively or recursively).

```
// write the reverseListPrint function here
```

10. Write a recursive function to add up all the even numbers in a Singly Linked List. Also, create a helper function to "kickoff" the recursion.

example: 11 → 18 → 15 → 4 → NULL Result : 22

BONUS Big-O (very tricky)

1)

```
for(int x{}; x < N; N /= 2, ++x)
  std::cout << "x: " << x <<  " N: " << N << "\n";
```

2)

```
for(int x = 100; x < 1203013; ++x)
  std::cout << "Hello, World!\n";
```

3)

```
do {
  k |= q;
  n -= (n % 1) ? 1 : 2;
} while(n > 0);
```

4)

```
for(;;){
  n &= 1; //bitwise and
  n--;
  if(n <= 0) break;
}
```

5)

```
for(int x{};;++x){
  if(x > n) break;
  x += 3;
  x -= 4;
  --n;
}
```

HELPFUL Big-O RESOURCES:

Singly-Linked List Analysis Summary

Operation	Time Efficiency
Default constructor (create empty)	O(1)
Size	O(1)
Front, Back	O(1)
AddFront, AddBack	O(1)
RemoveFront	O(1)
RemoveBack	O(n)
Get(i), Set(i, x)	O(n)
Add(i, x), Remove(i)	O(n)

Doubly-Linked List Analysis Summary

Operation	Time Efficiency
Default constructor (create empty)	O(1)
Size	O(1)
Front, Back	O(1)
AddFront, AddBack	O(1)
RemoveFront	O(1)
RemoveBack	O(1)
Get(i), Set(i, x)	O(n)
Add(i, x), Remove(i)	O(n)

Vector Analysis Summary

Operation	Time Efficiency
Default constructor (create empty)	O(1)
Size()	O(1)
Get(i)	O(1)
Set(i, x)	O(1)
Add(i, x)	O(n)
Add(Size(), x) (add to back)	O(1) amortized

Stack Analysis Summary

Operation	List-Based Stack	Vector-Based St
Default constructor (create empty)	O(1)	O(1)
Size	O(1)	O(1)
Top	O(1)	O(1)
Push(x)	O(1)	O(1) amortized
Pop	O(1)	O(1) amortized